AndroNet: Privilege-Adaptive Packet Capture and On-Device Multi-Layer Threat Detection for Android

Swastik✉, Ritik, Kamal Akhter, and Syed Misbah Uddin

Department of Computer Science and Information Technology,
Central University of Jammu, Samba 181143, Jammu and Kashmir, India
swastik362004@gmail.com, ritikrajput147@gmail.com,
kamalakhter694@gmail.com, bytesyed@gmail.com

Abstract. Security analysis of phone traffic still tends to end with a capture file on a desktop. Android tools can now capture without root, through a local VPN, or with root at the interface, yet attack detection has largely stayed off the device. AndroNet is an open-source analyser that feeds both capture paths into one canonical packet record and runs protocol dissection and three detection layers on the handset: volumetric detectors, signature matching, and stateful multi-packet rules. We evaluate it on a Pixel 8 with 28 public attack captures held out until the code was frozen, and with 1.01 million benign packets including the phone's own browsing. Measurement reshaped the system. The first implementation passed its unit tests yet flagged 48.8% of benign phone packets, missed a SYN flood outright, and analysed only 130 packets per second. Correcting the exposed defects cut flagged benign traffic from 33.9% to 0.022% of packets and per-packet cost to about 120 µs. Recall stayed at 12 of 19 in-scope held-out captures, with two detections gained and two lost. Through its VPN tunnel AndroNet sustains 39 Mbit/s downstream. A direct comparison with PCAPdroid, the leading open-source capture tool, shows the price of this design: 20% of its download rate and 39% more power under a 10 Mbit/s load. The rooted path, exercised on an emulator, also detects inbound scans and ARP spoofing, which no VPN path can see.

Keywords: Mobile network security · Intrusion detection · Android VpnService · Packet capture · On-device analysis.

1 Introduction

Ask a network analyst to examine suspicious traffic and the answer is reflexive: open Wireshark. Ask for the same from the phone in their pocket and the reflex fails, because packet analysers assume things Android withholds from an ordinary app: promiscuous interface access, raw sockets, and generous CPU and memory headroom. A penetration tester carrying Kali NetHunter [11], an Android build of the Kali toolchain, often still tethers the handset to a laptop for packet-level visibility, spending the mobility that motivated the platform on the first move.

Capture itself is no longer the obstacle. The VpnService API [2] lets an unprivileged app route device traffic through a local tunnel [15, 10, 30], and PCAPdroid [7], the best-known open-source tool in this class, offers that path alongside a root mode that reads

the interface directly. The harder question is what happens to the traffic next. PCAPdroid can flag connections to hosts on third-party blocklists [8]. Payload signatures and multi-packet behaviour remain the province of systems such as Snort [18] and Zeek [12], which on a phone need a rooted NetHunter chroot. Chen et al. [5] built an Android intrusion prevention system on VpnService and tcpdump with signature databases, but we have found neither an open implementation of that approach nor per-packet cost and energy measurements for it on a handset.

AndroNet addresses this gap. Its root-free and rooted back-ends emit one canonical packet record, and a single pipeline above that record performs protocol dissection and three detection layers, ordered from cheap and broad to costly and specific. The whole system runs on the phone with no server or tethered host.

This paper also reports what happened when that design met real traffic. Our first implementation had a passing unit test suite and looked sound on paper. Its first run on a phone flagged over 40% of ordinary packets, missed a 400-packet-per-second SYN flood, and took 7.7 ms per packet where a design-time estimate had promised two or three. Each failure traced to a defect that only appears when stages are joined, or only on the target runtime. We corrected them against a development set, froze the code, and only then measured on third-party captures.

Concretely, this paper contributes:

1. **One analysis path for two capture back-ends.** A root-free VpnService path and a rooted AF_PACKET path share one record builder and one pipeline. Across 22 shared attack scenarios they raise identical alert sets (Sect. 6.4).
2. **An on-device detection engine** whose behavioural rules count qualifying packets per source, with limits calibrated on measured phone traffic (Sect. 5).
3. **A measured evaluation on a physical phone**: held-out accuracy, per-layer cost on each core class, throughput, energy, and a direct comparison with PCAPdroid (Sect. 6). Code, harness and result files are public [26].
4. **A catalogue of the defects that measurement exposed**, with their measured effects, as guidance for anyone building detection for this platform (Sect. 6.2).

2 Background and Related Work

2.1 Capturing Traffic on Android

Android applications are sandboxed and cannot open raw sockets. The standard work-around declares the app a VPN, receives outbound traffic on a TUN descriptor, and relays it through sockets the app owns. Haystack [15], later Lumen [14], used this for in-situ study of app traffic, while AntMonitor [10] and MopEye [30] applied it to monitoring and per-app performance measurement. Each flow is terminated and re-originated at a local endpoint, with two lasting consequences: inbound packets are synthesised, so payloads are genuine while low-level header fields are not the bytes that crossed the network [8], and only connections started by the device enter the tunnel.

PCAPdroid [7] is the closest system to ours. It relays traffic with the zdtun library [9], which AndroNet also uses, classifies flows with nDPI [6], and can decrypt TLS through a mitmproxy add-on. On a rooted device it spawns a capture daemon and continues to

analyse what it captures [8], so a choice between capture paths is already behind a single interface in that tool. AndroNet's difference lies above the capture layer, as Table 8 sets out.

2.2 Detecting Attacks in Traffic

Snort [18] established signature matching against curated rule sets, and Bro, now Zeek [12], stateful analysis of network events. Volumetric attacks are the classic province of thresholds, whose trade-offs among detection probability, false alarms and delay are well mapped [22]; dynamic thresholds reduce false positives further in software-defined networks [13]. These systems run on servers and appliances. PCAPdroid's paid malware detection matches hosts against third-party blocklists, and its documentation is candid that this is no complete solution [8].

Detection on the phone itself has precedents. AndroIDS [19] brought Snort-style signatures to Android in 2013 through an iptables queue, which needs root, and left statistical anomaly detection to a server; its slides report no measurements, and we could not find its code. Commercial products detect network attacks on the device [31], but they are closed and we found no accuracy or per-packet cost for them. Small open prototypes such as Syrrak [16] and Breakline [1] apply rate and flow rules to VpnService traffic from packet headers alone, and we found no evaluation of them.

Methodology matters as much as mechanism. Attacks are rare, so the false alarm rate largely decides whether a detector is usable [3], and benchmark datasets have been criticised for ageing and inconsistent construction [17, 21]. We therefore report false alarms on a real phone's traffic, hold third-party captures out of development, and publish results per capture.

3 Threat Model and Design Goals

AndroNet runs on the device whose traffic it observes, operated by the owner or an authorised assessor. It monitors and alerts; it does not block or rewrite connections. We consider an adversary whose activity shows in the device's own traffic: malware on the device beaconing to command-and-control infrastructure or exfiltrating data, an attacker on the same network segment scanning, flooding or manipulating ARP, and hosts exhibiting recognisable attack tooling. An adversary with kernel-level control of the device defeats any analyser on it and is out of scope, as is activity confined to encrypted payloads with no distinguishing header, size or timing signal.

Four goals order the design. **G1, universality:** useful analysis on unmodified consumer devices, with root adding visibility. **G2, capture-mode transparency:** no component above the capture tier branches on capture mode. **G3, on-device detection:** detection completes on the handset within a per-packet budget compatible with sustained use. **G4, interoperability:** output stays readable by the desktop toolchain, so AndroNet complements established forensic workflows.

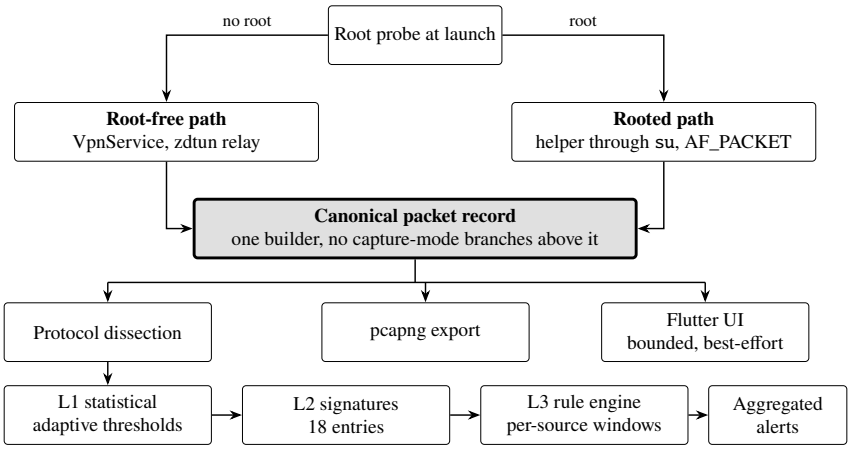


Fig. 1. AndroNet's architecture. The canonical packet record is the abstraction boundary: nothing below it knows which capture path produced a packet, which is what goal G2 requires.

4 System Architecture

AndroNet has three tiers: a Flutter interface, a Kotlin service layer and a C native layer (Fig. 1). At launch it probes for a working `su`. A rooted device selects the root path and any other device the VPN path, and the user can override the choice. Both paths build the same record, defined in one source file, and hand it to one pipeline.

4.1 The Canonical Packet Record

Each captured packet is normalised into one record: source and destination address and port, transport and application protocol, size and timestamp, TCP flags, the owning app where the platform can resolve it, capture mode and direction, and for ARP the sender's address and MAC. The capture mode is carried for provenance only: no analysis component reads it, and a unit test asserts that both modes raise identical alerts for identical bytes. Keeping the builder in one place has practical weight, as Sect. 6.2 shows: most of our worst defects were disagreements between a stage's output and the next stage's expectation.

4.2 The Two Capture Paths

Root-Free Path. AndroNet declares a VpnService, obtains a TUN descriptor and relays traffic through zdtun [9]. Only the user-granted VPN permission is needed, and connectivity is preserved because packets are forwarded after inspection. zdtun is not thread-safe, so a single loop owns it: the loop drains the TUN descriptor, then lets zdtun service its sockets, and it sleeps only when a `select` over both reports nothing to do.

Rooted Path. An app process cannot open a packet socket whatever the device's root status, and Android forks apps with an empty capability bounding set, so even a setuid helper could not regain `CAP_NET_RAW`. AndroNet therefore starts a small helper through `su`. The helper opens one AF_PACKET socket, drops to the app's user ID, which the open socket survives, and streams frames over standard output. It exits as soon as the app stops reading, so a crashed app leaves no capture process behind. The `SOCK_DGRAM` socket type strips the link layer, so one parser serves every interface type and ARP arrives ready to parse.

Export and Interface. Both paths hand the analysis stage packets that begin at the IP header, and AndroNet exports them as pcapng [28] that Wireshark and tshark read directly, with a high volumetric anomaly score carried as a standard packet comment (checked on a 2,747-packet export from the Pixel 8). A traffic burst can outrun the interface thread by an order of magnitude, so the stream to the UI is bounded and best-effort, while analysis and export still see every packet.

4.3 Protocol Dissection

For HTTP on any port the dissection stage extracts the request line, a percent-decoded URL, status and headers, and a bounded flow table recognises request bodies that travel in their own segment. For DNS it reads the question, record types, answers and transaction identifiers. For TLS it reads the record-layer version, handshake type and Server Name Indication from the cleartext handshake, with no key material or interception. It also parses DHCP, QUIC, SIP, RTP, SMB and NTP, and a port-based classifier names about forty application protocols. Payloads on encrypted ports are marked and withheld from later payload predicates.

5 Three-Layer Detection Engine

Every enriched packet traverses three layers in sequence, ordered as a cost gradient. All detection is confined to the packet, plus small bounded per-source state. Findings are aggregated: repeats of one finding about one host within 10 s yield a single alert, as with event filtering in Snort.

5.1 Layer 1: Statistical Detectors

Table 1 lists the detectors. Thresholds adapt to the network. A 60 s learning period, during which alerts are muted, records the per-second peak of each counter. The threshold becomes 2.5 times the 90th percentile of those peaks, clamped between a floor and a ceiling, where the ceiling is the larger of the default and four times the learned baseline and stays fixed once learning ends. An exponential moving average ($\alpha = 0.05$) then tracks the baseline, but a sample that reached the threshold is a detection and never trains it, so an attack cannot raise the bar it is measured against.

Table 1. Layer 1 detectors. Floors and defaults come from measured benign traffic (Sect. 6.1).

Detector	Trigger	Severity
Port scan	destination ports from one source in 10 s (floor 8, default 20)	High
SYN flood	SYN packets to one target per second (floor 30, default 100)	Critical
Connection flood	new connections from one source per second (floor, default 60)	High
DNS tunnelling	query name longer than 50 characters	Medium
ARP spoofing	observed IP-to-MAC binding changes (rooted path)	Critical
Malformed TCP	SYN set together with FIN or RST	Medium
Covert channel	high entropy in ICMP, or outgoing HTTP or DNS, for 5 packets	High+

5.2 Layer 2: Signature Database

The second layer matches 18 signatures across eight categories: command and control (4), web attacks (3), reconnaissance (3), malware (2), brute force (2), network attacks (2), exploits (1) and exfiltration (1). A signature binds an identifier, category and severity to a pattern built from seven constructors: `PayloadContains`, `HeaderPattern` (a regular expression on a named HTTP field), `PortPattern`, `IpPattern`, `DnsPattern`, `UrlPattern`, and `CompositePattern`, which closes the set under AND and OR. Entries include Meterpreter method names, the default Cobalt Strike User-Agent, Nmap, Nikto and sqlmap fingerprints, login-tool banners, the WannaCry kill-switch domains and the EternalBlue echo string. Two entries, the Tor exit and known-C2 lists, ship with a handful of sample indicators only: operational use needs a threat feed, which is future work.

5.3 Layer 3: Rule Engine

The third layer evaluates 10 rules (Table 2) over a grammar of 25 predicates and the combinators And, Or and Not. Predicates cover packet headers, payload content, HTTP and DNS fields, and three temporal forms: packet rate, unique destinations, and time since another rule fired.

The temporal predicates carry the design. Evaluation has two passes. The first ignores rate predicates, and a packet that fails it is not the kind this rule counts, so it never enters the rule's window. A packet that passes joins a window kept per rule and per source, and the second pass evaluates the rate predicates over that window alone. "More than 10 per second" thus means ten suspicious packets from this source. Windows are bounded to 1000 entries for each of 64 sources per rule.

Worked Example. DNS tunnelling appears twice, at different points on the sensitivity curve. Layer 1 fires on a single long query name, which catches a slow tunnel at the risk of flagging a legitimately long name. The Layer 3 rule fires only when protocol, port, payload shape and sustained rate all agree. In the live runs, a tunnel at 25 queries per second raised both alerts, while one at a query every 2 s raised only Layer 1.

Table 2. The ten composite rules. Rates count qualifying packets per source.

Rule	Condition	Severity
Horizontal scan	TCP, ACK clear, over 100 unique destinations in 10 s	High
SQL injection	HTTP request, SQL injection pattern in URL or body	Critical
File download	GET of an executable, script or APK	High
DNS tunnelling	UDP/53, large payload or long encoded label, over 10/s	High
Brute force	SYN to SSH, RDP, FTP or Telnet, over 10 per minute	High
Cryptomining	stratum or XMRig login method in TCP payload	Medium
Bulk upload	outgoing, payload over 1 KiB, over 500 per 10 s	Medium
Shellcode	NOP sled or known shellcode prefix in cleartext payload	Critical
Scanner agent	bot, scanner, crawler, Python, curl or tool User-Agent	Medium
ICMP tunnelling	ICMP, payload over 64 bytes, over 10/s	High

6 Evaluation

We ask four questions. How accurate is detection on traffic no development decision saw? Do the two capture paths behave alike? What does analysis cost on real hardware? How does the whole system compare with PCAPdroid in throughput and energy?

6.1 Setup and Data

Hardware. The phone is a Pixel 8: Tensor G3 with one Cortex-X3 at 2.91 GHz, four Cortex-A715 at 2.37 GHz and four Cortex-A510 at 1.70 GHz, 8 GB RAM, Android 17 (unrooted user build), on 802.11ax Wi-Fi with the lab laptop on the same access point. Android's stock user build cannot exercise the rooted path, so we run it on an Android 16 x86-64 emulator image with root.

Harness. Replay experiments run the production Kotlin classes under ART on the phone, through `app_process`. The only source change is a mechanical substitution of the clock, so that sliding windows follow trace time; for version 1.3.0, whose packet parser lived inside an Android service, we also copied that parser verbatim into the harness. Version 1.4.0 gave identical verdicts on the phone and a desktop JVM on all 40 held-out traces (1,119,888 packets), despite different regular expression engines. Version 1.3.0 is too slow to finish the largest traces on the phone, so we report its desktop run, whose verdicts matched the phone on the 35 traces small enough to compare. Live experiments use a release build that differs from a normal build only in accepting scripted start and stop commands.

Data. Table 3 summarises the split, which we fixed before any detector saw the files. Development traffic informed every correction and limit. Its attack traces come from real tools where our lab allowed it (nmap and sqlmap), real protocol exchanges over sockets (for example stratum logins, and Nikto's request shape, because the laptop's antivirus blocks the tool itself), and crafted frames where a desktop socket cannot emit the packet. Limits in Tables 1 and 2 sit above what the phone session we had at that point reached: 80 distinct destinations per 10 s and 45 new connections per second over 131 s. The code was then frozen, with file hashes recorded [26], before any held-out

Table 3. Evaluation data. Attack sources: Wireshark [29], Sanders [20], StopDDoS [23], Bousseaden [4], Zeek [27] and a WannaCry capture by D. Szili [24].

Split	Traffic	Captures	Packets
Development	lab attack traces	32	54,198
	tcpreplay smallFlows [25], Pixel 8 browsing	2	84,556
Held out	third-party attack captures	28	108,168
	third-party benign captures, including bigFlows	11	810,700
	Pixel 8 browsing of 20 other sites, after the freeze	1	201,020

capture was replayed. After the held-out results we found that a script fault had kept only the first rotated file of each phone session. Over its full 391 s the development session reaches 126 destinations and 118 connections per second, above two limits. We left the limits frozen, since changing them now would void the split, and replayed the held-out session in full. Held-out attacks all come from third parties. We labelled them from their publishers' descriptions and wrote the expected alerts down in advance. Nine of them represent threats no AndroNet detector targets; they stay in the set so that results show what is missed.

6.2 What Measurement Changed

Version 1.3.0, our first implementation, passed all 106 of its unit tests. Table 4 lists what its first measurement exposed. Two patterns stand out. Unit tests built their input records by hand, so the dissector and the detectors could each pass while disagreeing on field names, which disabled every URL and User-Agent signature on real traffic. And several failures only exist on the target: ICU's regular expression engine turned patterns with leading and trailing wildcards into a 7.2 ms per-packet cost, and real page loads reach connection counts our desktop traces never showed. We now run real wire bytes through the whole pipeline in a contract test suite, part of the 145 tests that version 1.4.0 passes.

6.3 Detection Accuracy

We report *capture recall*, the share of in-scope attack captures raising at least one expected alert; the *packet false positive rate*, the share of benign packets that raise any alert; and *alert precision*, alerts on attack traffic over all alerts. Precision depends on the base rate of the data at hand, so we also give results per traffic source. Table 5 compares the versions on the held-out set.

The two versions detect the same number of captures, though two of them differ, and the equality flatters the first. Version 1.3.0 "detected" SSH password guessing by alerting on every packet on port 22, flagging 93% of benign SSH just the same, and WannaCry by alerting on nearly every packet on port 445. It also caught the spoofed ICMP flood, through the same all-hosts counting behind its false alarms, a detection that per-source windows give up. It missed the SYN flood and the path traversal that version 1.4.0 catches, and on 1.01 million benign packets it raised 352,310 alerts, an unusable volume.

Table 4. Defects in version 1.3.0 that measurement exposed, all corrected in version 1.4.0.

Defect	Measured or observed effect
Dissector wrote `uri`, `userAgent`; detectors read `url`, `User-Agent`	URL and User-Agent signatures and rules never fired
Thresholds sampled per packet, counters still rising	400 pps SYN flood raised no alert
Learning period did not run the counters	Thresholds always at the floor
Rate rules counted all traffic from all hosts	95,673 false alerts on the phone session alone
Two-byte shellcode pattern tried on ciphertext	2,128 critical false alerts on the phone session
Port-only signatures; OR where AND was meant	93% of benign SSH packets flagged
`<!DOCTYPE` treated as an XXE indicator	Alerts on every HTML page over HTTP
Quadratic regular expressions under ICU	7.7 ms mean per packet, 130 pps
Five predicates fell through to `else -> false`	Those conditions could never hold
Null stored in a map for unresolved app owners	Packets forwarded but never analysed
zdtun driven from two coroutines at once	15 Mbit/s tunnel, with the other fixes already in
Root capture absent from builds, bypassed analysis	No rooted path existed in practice

Version 1.4.0 raised 218. The bigFlows enterprise capture accounts for 180, and its largest groups are plain false alarms: 66 from 52 clients polling a web service whose path ends in `.dll`, which the download rule takes for a file; 35 from large uploads; and 30 from one server taking up to 154 connections per second from about 40 clients. Two benign IRC captures raised 34, of which 24 are ordinary chat commands matching our bot signature on the IRC port, and an SMB capture raised one ARP alert. On the phone’s own traffic version 1.4.0 raised three in replay, one connection flood and two bulk uploads, the same three as the live 9-minute session behind that capture (21 per hour).

Recall stands at 63%, and each miss has a specific cause in the capture. In the spoofed ICMP flood, 9,889 packets arrive from 9,886 source addresses, so no per-source window fills. In the ARP poisoning capture, the forged binding is the first one seen for its address, so no binding ever changes, even though one MAC claims two addresses. One capture we labelled a DNS channel from its published name turns out to be a cleartext Windows shell on TCP port 53, which our DNS detectors never examine; we kept the label as written in advance. Another channel carries PowerShell commands in TXT responses, with query names under 28 characters at about one query per second. A Meterpreter session’s 4.9 MB measures 7.99 bits of entropy per byte, so no method name shows in the clear. Nmap’s version scan touches two ports against a floor of eight, and SSH guessing in the Zeek capture peaks at exactly ten attempts per minute, where the rule requires more than ten. We tuned nothing after seeing these results, and each is a candidate for future work. Of the nine captures no detector targets, one raised an alert, the shellcode rule on an MS04-011 exploit, which we leave uncounted. Real worm traffic fared better: WannaCry spreading through EternalBlue raised the signature we wrote from the published echo string, with no sample in hand during development.

We also ran 22 attack scenarios live on the Pixel 8 through the real tunnel, with the app’s alert log as the only output: all 22 raised their expected alerts. These scenarios

Table 5. Held-out detection results by version, and by layer for version 1.4.0. A capture counts for a layer if that layer raised an expected alert on it.

Measure	v1.3.0	v1.4.0	
Capture recall (in scope)	12/19	12/19	
Packet false positive rate	33.9%	0.022%	
Alerts on benign traffic	352,310	218	
Alerts per 1000 benign packets	348	0.22	
Alert precision	19.0%	20.7%	
Phone browsing: packets flagged	48.8%	0.001%	
v1.4.0 by layer	**L1**	**L2**	**L3**
Captures detected	7	4	3
Alerts on attack traffic	25	18	14
Alerts on benign traffic	70	31	117
Alert precision	26.3%	36.7%	10.7%

Table 6. What each capture path can observe, as measured. Encrypted payloads expose only headers, sizes, rates and the TLS handshake on either path.

Traffic	VPN path	Rooted path
Scans, floods and tunnels from software on the device	yes	yes
Cleartext web attacks, C2 and tool fingerprints from the device	yes	yes
Inbound scans and floods from the LAN	no	yes
ARP spoofing, DHCP and other link-level traffic	no	yes
Verbatim inbound packet headers	no	yes

resemble the development traces, so this verifies the path end to end and is no independent accuracy claim.

6.4 Capture-Path Parity and Observability

Running the same 22 scenarios under the rooted path gave identical alert sets, which is direct evidence for goal G2. Injecting 7 further traces as real frames on the interface, covering inbound SYN, FIN, NULL and Xmas scans, a SYN flood and ARP spoofing, raised the expected alert in every case. The helper exited within 5 s of the app being killed. These rooted runs used an emulator and a stand-in for `su`, which is a limit of this evidence: we have not yet run the rooted path on a physical rooted handset.

Identical analysis does not mean identical visibility, and Table 6 states the difference. To confirm it, we scanned the phone with real nmap from the laptop across Wi-Fi during VPN-mode capture. Over three scans of 1000 ports each, the phone's kernel answered 3,000 probes with resets, so the packets did arrive, and AndroNet raised no scan alerts, because unsolicited inbound packets never enter a VPN tunnel. Any VPN-based tool shares this blind spot, and it is the strongest reason to keep a rooted path.

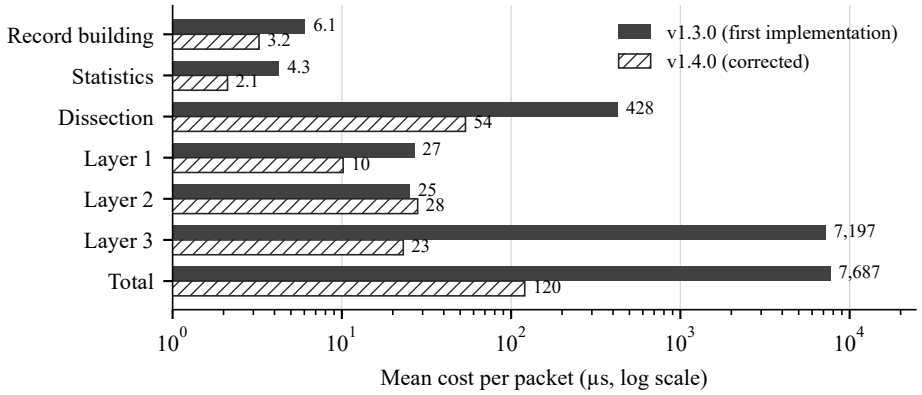


Fig. 2. Mean per-packet cost by stage on a Cortex-A715 core under ART, replaying tcpreplay smallFlows. The axis is logarithmic.

6.5 Per-Packet Cost

We cost each layer by ablation: replaying a trace with the signature and rule lists emptied by reflection, so that the production code carries no timers. Figure 2 shows the result for both versions on a mid core. The rule layer fell from 7.2 ms to 23 µs per packet and dissection from 428 to 54 µs. To isolate the causes we rebuilt version 1.3.0 with single changes. Removing only the leading and trailing wildcards from its rule expressions, which cannot alter a containment match, brings the rule layer to 291 µs. Replacing only its per-byte hexadecimal formatting brings dissection to 91 µs. Version 1.4.0 averages 120 µs per packet (35 at the median, 319 at p95), which sustains about 8,300 packets per second on one mid core, 12,300 on the big core and 2,300 on a little core. On the phone's own capture, whose encrypted payloads skip the payload predicates, the mean is 25 µs; version 1.3.0 took 2,234 µs over the first 3,000 packets of that capture. Detection state occupies 444 KiB of heap after replaying the trace; version 1.3.0, which retained whole packet records in its rule history, used 3,396 KiB.

6.6 Throughput, Energy and PCAPdroid

Table 7 compares no capture, PCAPdroid 2.0.2 and AndroNet 1.4.0 on the same phone and network, with conditions shuffled in each of five rounds. Figures are medians of 50 MB transfers, two per direction in each round. For energy, a script on the phone logged battery current and voltage once per second over 150 s, with the cable out, the screen on at minimum brightness and three repetitions in a rotating order. Current is read from the fuel gauge, so these figures are coarser than those from an external power monitor. We measured PCAPdroid's free build with no capture file written, and did not test its paid blocklist feature.

PCAPdroid is the faster relay: on this link it keeps pace with no capture at all, and most of the gap exists before any analysis runs, since forwarding alone reaches 37% of

Table 7. Head-to-head on the Pixel 8. Throughput and connect time are TCP between phone and laptop. AndroNet's columns are forwarding alone, forwarding with analysis, and the full app with its UI stream. Power is whole-device, under a 10 Mbit/s download and at idle, with its standard deviation across repetitions.

Measure	No capture	PCAPdroid	AndroNet		
			forward	analysis	full
Download (Mbit/s)	190	189	70	42	39
Upload (Mbit/s)	206	194	105	75	47
TCP connect (ms)	22.8	34.9	28.5	30.1	30.0
App CPU (% of a core)	–	19	41	64	128
Peak memory (MiB)	–	154	299	424	414
Power, 10 Mbit/s (W)	1.10 ± 0.04	1.14 ± 0.03			1.59 ± 0.04
Power, idle (W)	0.86 ± 0.14	0.88 ± 0.02			0.96 ± 0.18
Battery, 10 Mbit/s (%/h)	7.1	7.1			9.4
Battery, idle (%/h)	5.6	5.1			5.4

its download rate while using only 41% of a core. Raw CPU is therefore not the limit. PCAPdroid drives the same zdtun library with a 10,000-byte tunnel MTU and blocking tunnel writes, where ours uses 1500 bytes and non-blocking writes; we have not yet isolated how much of the gap each explains. Analysis takes the download to 42 Mbit/s, and the UI stream then costs mainly CPU, which it doubles. For scale, the 9-minute browsing session, relayed through AndroNet itself, averaged 2.4 Mbit/s with a busiest second of 63 Mbit/s, so ordinary browsing sits well inside this capacity on average while its bursts reach it. Under the 10 Mbit/s load AndroNet adds 0.49 W to the no-capture baseline, where PCAPdroid adds 0.05 W, and its process stays near 100% of a core against 4%. At idle the three differ by less than the spread between repetitions. Two idle runs in which the cable was reconnected early are excluded.

Table 8 states the novelty plainly. PCAPdroid is the stronger capture and visibility tool: broader classification, TLS decryption, a faster relay. Combining both capture paths behind one pipeline is common ground. What AndroNet adds is the combination: payload signatures, stateful multi-packet rules and volumetric detectors together, on an unrooted handset that stays online, in open source, with accuracy, cost and energy measured on that handset. Each ingredient has precedents (Sect. 2), and we found no other system with all of them. We make no claim to a new detection algorithm; the statistical, signature and rule paradigms are long established.

6.7 Threats to Validity

Our held-out attack set is small, and many captures are short, so recall carries wide uncertainty; we publish per-capture results for that reason. Many third-party traces come from desktop networks. The benign phone traffic is scripted browsing on one device, over Wi-Fi, for minutes, and real use spanning days and apps may raise more false alarms. Replaying traces at the IP layer omits VPN-path effects on inbound headers, although those do not affect payload or rate predicates. Limits were set on one phone

Table 8. Capabilities compared, from each tool's documentation and our measurements. • present, ○ partial or conditional, – absent.

Capability	Wireshark	Snort	PCAPdroid	AndroNet
Runs natively on Android	–	○[a]	•	•
Works without root	–	–	•	•
One analysis path over both capture paths	n/a	n/a	•	•
Application-layer dissection	•	•	•	○[b]
TLS decryption	○[c]	–	•	–
Reputation blocklists	–	•	○[d]	○[e]
Payload signatures	–	•	–	•
Stateful multi-packet rules	–	•	–	•
Volumetric attack detection	–	○	–	•
Connection blocking	–	•	○[d]	–

[a] in a rooted NetHunter chroot. [b] narrower than nDPI. [c] with supplied keys.
[d] paid feature. [e] sample lists only.

and may not suit others. Wi-Fi capacity varied between rounds, so small throughput differences mean little. Energy readings come from a single device's fuel gauge.

7 Discussion and Limitations

Encrypted traffic. Most phone traffic is TLS or QUIC, and AndroNet deliberately withholds such payloads from matching. Its few false alarms on phone traffic are partly a consequence: it sees handshakes, names, sizes and rates there. Payload signatures earn their place on cleartext traffic, on tool-generated traffic that often remains cleartext, and on a rooted assessment device. A malicious app using TLS on port 443 evades the payload layers entirely, leaving volumetric detectors, SNI and DNS names.

No stream reassembly. Detection works on packets. A signature split across TCP segments, or a query name below the length limit, escapes it. Reassembly would cost memory that the design has so far avoided.

Future work. Running on a physical rooted phone comes first, followed by threat-feed integration, a faster relay, and detectors for the misses above: flood counting that tolerates spoofing, ARP checks for one MAC claiming several addresses, inspection of DNS responses, and a check that traffic on a well-known port speaks that port's protocol.

8 Conclusion

AndroNet carries packet capture on Android through to detection on the handset, with one pipeline over a root-free and a rooted capture path. The more instructive result is how far its first implementation sat from its design. Passing unit tests and a plausible cost analysis coexisted with detectors that could not fire, thresholds that an attack could outrun, and a per-packet cost several times the estimate. Measurement on a real phone found what none of that revealed. After correction, the frozen system detects 12 of 19

in-scope held-out captures, flags 0.022% of benign packets, and analyses a packet in about 120 µs on a mid core. Our evaluation makes its limits equally plain: a slower relay than PCAPdroid, real misses on spoofed floods and DNS-borne commands, and a rooted path still awaiting physical hardware. For on-device security tools, an honest number from the phone is worth more than a careful estimate from the desk.

Acknowledgments. During the preparation of this work the authors used Claude (Anthropic; model Claude Opus 5.5, through Claude Code) in all sections of this manuscript. The tool drafted and revised the text, wrote the evaluation harness, traffic generators and scoring scripts, proposed and implemented the code corrections described in Sect. 6.2, produced the figures and tables from measured results, and helped operate the experiments on the authors' hardware. All measurements come from running the published code. The authors reviewed the manuscript and the code, verified the results, and take full responsibility for the content of this publication.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. aditya-p02: Breakline: Adaptive mobile cyber-defense engine. https://github.com/aditya-p02/Breakline, last accessed 2026/11/03
2. Android Developers: VpnService API reference. https://developer.android.com/reference/android/net/VpnService, last accessed 2026/11/02
3. Axelsson, S.: The base-rate fallacy and the difficulty of intrusion detection. ACM Transactions on Information and System Security **3**(3), 186–205 (2000). https://doi.org/10.1145/357830.357849
4. Bousseaden, S.: PCAP-ATTACK: Samples of post-exploitation techniques. https://github.com/sbousseaden/PCAP-ATTACK, last accessed 2026/11/02
5. Chen, G., Zhou, K., Peng, Y., Zhou, L., Zhang, Y.: A novel network intrusion prevention system based on android platform. International Journal of Internet Protocol Technology **14**(2), 67–76 (2021). https://doi.org/10.1504/IJIPT.2021.116254
6. Deri, L., Martinelli, M., Bujlow, T., Cardigliano, A.: ndpi: Open-source high-speed deep packet inspection. In: 2014 International Wireless Communications and Mobile Computing Conference (IWCMC). pp. 617–622. IEEE (2014). https://doi.org/10.1109/IWCMC.2014.6906427
7. Faranda, E.: PCAPdroid: No-root network monitor, firewall and PCAP dumper for android. Version 2.0.2, https://github.com/emanuele-f/PCAPdroid, last accessed 2026/11/02
8. Faranda, E.: PCAPdroid user guide. https://emanuele-f.github.io/PCAPdroid/, last accessed 2026/11/02
9. Faranda, E.: zdtun: Lightweight TCP/UDP/ICMP tunnelling library. https://github.com/emanuele-f/zdtun, last accessed 2026/11/02
10. Le, A., Varmarken, J., Langhoff, S., Shuba, A., Gjoka, M., Markopoulou, A.: Antmonitor: A system for monitoring from mobile devices. In: 2015 ACM SIGCOMM Workshop on Crowdsourcing and Crowdsharing of Big (Internet) Data. pp. 15–20. ACM (2015). https://doi.org/10.1145/2787394.2787396
11. OffSec: Kali NetHunter documentation. https://www.kali.org/docs/nethunter/, last accessed 2026/11/02
12. Paxson, V.: Bro: A system for detecting network intruders in real-time. Computer Networks **31**(23–24), 2435–2463 (1999). https://doi.org/10.1016/S1389-1286(99)00112-7

13. Rahouti, M., Xiong, K., Ghani, N., Shaikh, F.: SYNGuard: Dynamic threshold-based SYN flood attack detection and mitigation in software-defined networks. IET Networks **10**(2), 76–87 (2021). https://doi.org/10.1049/ntw2.12009
14. Razaghpanah, A., Nithyanand, R., Vallina-Rodriguez, N., Sundaresan, S., Allman, M., Kreibich, C., Gill, P.: Apps, trackers, privacy, and regulators: A global study of the mobile tracking ecosystem. In: 25th Network and Distributed System Security Symposium (NDSS). Internet Society (2018). https://doi.org/10.14722/ndss.2018.23353
15. Razaghpanah, A., Vallina-Rodriguez, N., Sundaresan, S., Kreibich, C., Gill, P., Allman, M., Paxson, V.: Haystack: In situ mobile traffic analysis in user space. arXiv:1510.01419 (2015)
16. rcallaby: Syrrak: On-device mobile intrusion detection system for Android. https://github.com/rcallaby/Syrrak, last accessed 2026/11/03
17. Ring, M., Wunderlich, S., Scheuring, D., Landes, D., Hotho, A.: A survey of network-based intrusion detection data sets. Computers & Security **86**, 147–167 (2019). https://doi.org/10.1016/j.cose.2019.06.005
18. Roesch, M.: Snort: Lightweight intrusion detection for networks. In: 13th USENIX Systems Administration Conference (LISA '99). pp. 229–238. USENIX Association (1999)
19. Sánchez, J.: Building the first Android IDS on network level. DeepSec 2013, slides, https://deepsec.net/docs/Slides/2013/DeepSec_2013_Jaime_Sanchez_-_Building_The_First_Android_IDS_On_Network_Level.pdf, last accessed 2026/11/03
20. Sanders, C.: Practical Packet Analysis. No Starch Press, San Francisco, 3rd edn. (2017), capture files: https://github.com/chrissanders/packets, last accessed 2026/11/02
21. Sarhan, M., Layeghy, S., Portmann, M.: Towards a standard feature set for network intrusion detection system datasets. Mobile Networks and Applications **27**(1), 357–370 (2022). https://doi.org/10.1007/s11036-021-01843-0
22. Siris, V.A., Papagalou, F.: Application of anomaly detection algorithms for detecting SYN flooding attacks. Computer Communications **29**(9), 1433–1442 (2006). https://doi.org/10.1016/j.comcom.2005.09.008
23. StopDDoS: Packet captures of DDoS attacks. https://github.com/StopDDoS/packet-captures, last accessed 2026/11/02
24. Szili, D.: PCAP of WannaCry spreading using EternalBlue. Malware-Traffic-Analysis.net guest post, 18 May 2017, https://www.malware-traffic-analysis.net/2017/05/18/index2.html, last accessed 2026/11/02
25. Tcpreplay project: Sample captures. https://tcpreplay.appneta.com/wiki/captures.html, last accessed 2026/11/02
26. The AndroNet authors: AndroNet: Source code, evaluation harness and result files. Version 1.4.0, https://github.com/amibhai/AndroNet, last accessed 2026/11/02
27. The Zeek Project: Zeek test suite traces. https://github.com/zeek/zeek/tree/master/testing/btest/Traces, last accessed 2026/11/02
28. Tuexen, M., Risso, F., Bongertz, J., Combs, G., Harris, G., Chaudron, E., Richardson, M.: PCAP next generation (pcapng) capture file format. IETF Internet-Draft draft-ietf-opsawg-pcapng, https://datatracker.ietf.org/doc/draft-ietf-opsawg-pcapng/, last accessed 2026/11/02
29. Wireshark Foundation: Sample captures. https://wiki.wireshark.org/SampleCaptures, last accessed 2026/11/02
30. Wu, D., Chang, R.K.C., Li, W., Cheng, E.K.T., Gao, D.: Mopeye: Opportunistic monitoring of per-app mobile network performance. In: 2017 USENIX Annual Technical Conference (USENIX ATC '17). pp. 445–457. USENIX Association (2017)
31. Zimperium: Zimperium for SOTI: Enabling complete end-to-end mobile threat protection. Solution brief (2020), https://lp.zimperium.com/hubfs/MTD/SB/GEN/SOTI_Zimperium_Solution_Brief.pdf, last accessed 2026/11/03